SBA301 – SLOT 25

CHAPTER 16 – SECURING REST SERVICES WITH SPRING BOOT

PART B: AUTHENTICATION & AUTHORIZATION • JWT IMPLEMENTATION IN SPRING REST

CORE: Tư duy trung tâm của Slot 25: Spring Security không chỉ là một `SecurityFilterChain` và JWT không chỉ là chuỗi token. Sinh viên phải nhìn được toàn bộ authentication pipeline: credential → Authentication → AuthenticationManager/Provider → UserDetailsService/PasswordEncoder → authenticated principal → JWT issuance; sau đó ở request kế tiếp: Bearer token → JWT validation → SecurityContext → authorization → protected Controller.

Thông tin	Giá trị
Môn học	SBA301 – Integrate Single Page Application with Spring Boot
Slot	25
Chủ đề	Chapter 16 Part B – Authentication & Authorization; JWT implementation in Spring REST
CLO chính	CLO4 – Apply Spring Boot REST security in an integrated application
Ngữ liệu xuyên suốt	Orchid Management / Lab 06 – User registration, login, JWT and protected user endpoints
Course baseline	JDK 21 • Spring Boot 3.x track • Spring Security 6.x style • Spring Data JPA • SQL Server • JJWT • Postman
Thời lượng gợi ý	135 phút trên lớp + implementation/self-study
Learning flow	Prepare → Understand → Trace → Implement → Break → Diagnose → Verify → Transfer

SCOPE: Scope lock: Slot 25 đi sâu implementation mechanics của Authentication/Authorization và JWT theo Chapter 16/Lab 06. Slot 24 Part A đã đặt nền security concepts/OAuth/JWT fundamentals. Slot 26 Part C sẽ hợp nhất role/JWT/CORS/negative tests. Vì vậy Slot 25 không biến thành CORS/negative-testing workshop và không dạy lại toàn bộ OAuth 2.0.

CÁCH SỬ DỤNG GIÁO TRÌNH

Phần IV là knowledge base lý thuyết có phân cấp sâu để sinh viên hiểu kiến trúc Spring Security và JWT implementation. Phần VI là workbook step-by-step bám Lab 06: tạo User/DTO/Repository, cấu hình PasswordEncoder/UserDetailsService/AuthenticationProvider, xây JWT service, filter, AuthenticationService, Controller, secure endpoints và test bằng Postman.

Pha	Sinh viên làm gì?	Bằng chứng
Prepare	Chạy REST baseline + DB; kiểm dependencies.	Baseline run
Understand	Map class/bean vào authentication architecture.	Security architecture map
Trace	Trace login + protected request.	Method chain
Implement	Tạo user/auth/JWT/filter/security config.	Source + build
Break	Sai password/key/token/filter order.	Failure evidence
Diagnose	Đọc 401/403/log/call stack.	Debug note
Verify	Signup/login/me/users qua Postman + DB.	Verification matrix
Transfer	So sánh custom JWT lab với Resource Server production.	Architecture worksheet

0. SINH VIÊN CẦN CHUẨN BỊ GÌ TRƯỚC KHI HỌC SLOT

0.1. Kiến thức cần ôn lại

- Slot 24 Part A: authentication vs authorization, JWT header/payload/signature, bearer token, OAuth/JWT distinction.
- Spring Boot dependency injection và bean lifecycle cơ bản.
- REST Controller, Service, Repository, Spring Data JPA và SQL Server.
- HTTP Authorization header và status 200/201/400/401/403/500.
- JPA Entity, unique email/username và CRUD repository.
- Hash khác encryption; password không được lưu plaintext.

0.2. Công cụ và project baseline

Thành phần	Yêu cầu	Kiểm tra
JDK	21	java -version
Maven	Wrapper hoặc Maven tương thích Boot 3.x	mvn -v / mvnw.cmd -v
IDE	IntelliJ IDEA	idea .
Dependencies	Web, Security, Data JPA, SQL Server, Validation, Test, JJWT	mvn dependency:tree
SQL Server	DB kết nối được	SSMS SELECT
Postman	Collection cho /auth/signup, /auth/login, /users/me	environment variables

Thành phần	Yêu cầu	Kiểm tra
Git	baseline sạch	git status

0.3. Pre-class 3–2–1 Learning Note

3 điều đã biết	2 điều còn mơ hồ	1 câu hỏi muốn kiểm chứng
JWT được ký; Spring Security dùng filter; role dùng cho authorization.	AuthenticationManager và Provider khác gì? UserDetails có phải Entity không?	Một login request thực sự đi qua những bean/method nào trước khi tạo JWT?

0.4. Pre-flight checklist

- Backend CRUD baseline compile/start thành công trước khi thêm security.
- SQL Server connection đúng và có database cho User.
- Không commit password DB hoặc JWT secret.
- Có Postman/HTTP client để test độc lập với React.
- Commit/tag gợi ý: s25-before-auth-jwt.

Baseline

```
mvn -q -DskipTests compile
mvn test
mvn spring-boot:run

# Windows Maven Wrapper
mvnw.cmd test
mvnw.cmd spring-boot:run
```

I. TÓM TẮT CÁC CHỦ ĐỀ & MỤC TIÊU TRONG BÀI HỌC

1.1. Chapter 16 Part B trong progression của môn học

Part A – Slot 24	Part B – Slot 25	Part C – Slot 26
Security concepts; OAuth/JWT fundamentals	Authn/Authz architecture + implement signup/login/JWT/protected endpoints	Consolidation: role/JWT/CORS/negative tests

1.2. Mục tiêu học tập

1. Phân biệt và liên kết Authentication, Principal, GrantedAuthority, Role và Authorization.
2. Giải thích SecurityContextHolder, SecurityContext và Authentication trong request lifecycle.
3. Giải thích vai trò AuthenticationManager, AuthenticationProvider, UserDetailsService và PasswordEncoder.
4. Thiết kế User entity/security representation không lưu password plaintext.
5. Tạo signup flow encode password trước khi persist.
6. Tạo login flow dùng AuthenticationManager thay vì tự so sánh password.
7. Tạo JWT có subject, issued-at, expiration và signature.
8. Validate/extract claims từ JWT ở request kế tiếp.
9. Xây JWT authentication filter và đặt Authentication vào SecurityContext đúng thời điểm.
10. Cấu hình SecurityFilterChain để permit auth endpoints và bảo vệ endpoint còn lại.
11. Tạo AuthenticationController và protected endpoints `/users/me`, `/users` theo course example.
12. Trace và kiểm chứng login/protected-request end-to-end bằng Postman, debugger và DB.

1.3. Hai workflow bắt buộc phải hiểu

```
A. SIGNUP
POST /auth/signup
 -> AuthController.signup
 -> AuthService.signup
 -> validate duplicate/email
 -> PasswordEncoder.encode(rawPassword)
 -> UserRepository.save(user)
 -> SQL INSERT
 <- safe response (NO password)
```

```
B. LOGIN
POST /auth/login
 -> AuthController.login
 -> AuthService.authenticate
 -> AuthenticationManager.authenticate
 -> DaoAuthenticationProvider
 -> UserDetailsService.loadUserByUsername(email)
 -> UserRepository.findByEmail
 -> PasswordEncoder.matches(raw, hash)
 <- authenticated principal
 -> JwtService.generateToken(principal)
 <- token response

C. PROTECTED REQUEST
GET /users/me
Authorization: Bearer <JWT>
 -> SecurityFilterChain
 -> JwtAuthenticationFilter
 -> extract/validate JWT
 -> resolve user / authorities
 -> SecurityContext.setAuthentication
 -> authorization
 -> UserController.me
 <- response
```

1.4. "Có JWT" chưa đủ

Mức	Biểu hiện	Rủi ro
Demo	Token generate được	Có thể chưa verify, password lưu sai, endpoint vẫn mở
Course-ready	AuthManager + PasswordEncoder + JWT filter + protected endpoints + verification	Đạt mục tiêu Part B
Production direction	Resource Server/issuer/JWK, lifecycle, rotation, observability, hardening	Giảm custom security code

II. TRỌNG TÂM CỐT LÕI & KỸ NĂNG SINH VIÊN CẦN NẮM VỮNG

2.1. Authentication – "Who are you?"

Authentication xác minh identity từ credential. Trong username/password flow, credential ban đầu là password; sau login, ứng dụng có thể đổi sang credential ngắn hạn như token.

2.2. Authorization – "Can you do this?"

Authorization chỉ diễn ra có ý nghĩa sau khi identity đã được xác lập. Nó dùng authorities/roles/policy để quyết định access.

2.3. Principal, GrantedAuthority, Role

Principal là identity hiện tại; GrantedAuthority là permission abstraction; Role là nhóm permission có semantic nghiệp vụ. Spring thường biểu diễn role thành authority có prefix `ROLE_`.

2.4. SecurityContextHolder / SecurityContext

SecurityContextHolder là nơi framework truy cập SecurityContext hiện tại; SecurityContext chứa Authentication của request.

2.5. Authentication object

Authentication có principal, credentials, authorities và trạng thái authenticated. Trước authentication nó là input; sau authentication nó đại diện current user.

2.6. AuthenticationManager

API điều phối authentication. Controller/Service không nên tự query user rồi so sánh password thủ công nếu đã dùng Spring Security architecture.

2.7. AuthenticationProvider

Provider thực hiện một authentication strategy. `DaoAuthenticationProvider` dùng UserDetailsService + PasswordEncoder cho username/password.

2.8. UserDetailsService

Contract load user theo username/email. Course source dùng email làm username vì email là thông tin duy nhất.

2.9. UserDetails

Security-facing user details gồm username/password/authorities/account status. Domain Entity có thể implements trực tiếp ở lab; production có thể dùng adapter để giảm coupling.

2.10. PasswordEncoder

Encode raw password khi signup; `matches(raw, encoded)` khi login. Không decode password. Dùng adaptive one-way hash như BCrypt/Argon2 direction.

2.11. JWT issuance

JWT chỉ nên tạo sau authentication success. Subject phải gắn identity đã xác thực, không lấy identity/role tùy ý từ request body.

2.12. JWT validation

Request filter phải verify signature và thời hạn trước khi tin subject/claims. Decode ≠ validate.

2.13. JWT Filter responsibilities

Đọc Bearer header → extract token → validate → resolve user/authorities → tạo Authentication → set SecurityContext → continue chain.

2.14. SecurityFilterChain

Khai báo endpoint nào public/protected và security policies. Auth endpoints thường permitAll; phần còn lại authenticated ở baseline.

2.15. Statelessness

JWT course flow thường cấu hình stateless session policy. Stateless không đồng nghĩa hệ thống không có DB/user/key/config state.

2.16. Error boundary

Bad credentials/missing token không nên biến thành 500. 401/403 được phân tách rõ hơn ở Slot 26 Part C.

2.17. Class-to-responsibility map

Class/Bean	Trách nhiệm	Không nên làm
User/AppUser	Persist identity + encoded password + role	Trả password ra JSON
UserRepository	Lookup user/email	Business authorization
UserDetailsService	Bridge repository → security user	Generate token
PasswordEncoder	encode/matches	Store raw password
AuthenticationProvider	Credential verification strategy	HTTP response formatting
AuthenticationManager	Orchestrate provider authentication	Persist entity
JwtService	Generate/parse/validate JWT	Query business data
JwtAuthenticationFilter	Convert valid bearer token → Authentication	Business CRUD
SecurityFilterChain	Security policies/filter wiring	Business rules
AuthService	Signup/login use cases	Direct servlet concerns
AuthController	HTTP boundary for signup/login	Password comparison

III. ỨNG DỤNG THỰC TIỄN KIẾN THỨC CỦA BÀI HỌC TRONG SẢN PHẨM THỰC TẾ

3.1. Orchid Management

Use case	Authentication	Authorization	Implementation focus
Register user	Public	Server controls default role	encode password + persist
Login	Credential verification	N/A	AuthenticationManager + JWT
View own profile	Bearer JWT	Authenticated	SecurityContext principal

Use case	Authentication	Authorization	Implementation focus
List users	Bearer JWT	Role/privilege direction	protected endpoint
CRUD orchid	Bearer JWT	Role policy direction	token → principal → endpoint

3.2. FUNewsManagement transfer

Assignment 02 có Admin và Staff. Kiến thức Part B dùng để thiết kế authentication pipeline chung: email/password → authenticated identity → token. Sau đó role Admin/Staff trở thành input cho authorization ở Part C/Assignment implementation.

Domain	Authentication concern	Authorization concern
SystemAccount	email/password; password hash	Admin vs Staff
NewsArticle	caller identity	Staff create/update/delete; public active read
Category	caller identity	Staff management + business delete rule

3.3. Production direction – custom JWT vs Resource Server

Lab 06 dùng custom JWT classes/filter để học mechanics. Current Spring Security hỗ trợ OAuth2 Resource Server cho JWT hoặc opaque bearer token; với JWT Resource Server, framework có thể verify signature, `exp`/`nbf`/`iss` và map scopes thành authorities. Đây là hướng production cần biết, nhưng không thay baseline Lab 06 giữa kỳ.

Course implementation	Production ecosystem direction
Custom JwtService	JwtDecoder / Resource Server
Shared secret local	Issuer/JWK/public key
Custom filter	BearerTokenAuthenticationFilter
Manual authority mapping	JwtAuthenticationConverter
App vừa issue vừa consume token	Authorization Server/IdP tách khỏi Resource Server khi cần

IV. NỘI DUNG CHI TIẾT – KNOWLEDGE BASE AUTHENTICATION, AUTHORIZATION VÀ JWT IMPLEMENTATION

READING GUIDE: Phần IV giải thích nền tảng, semantics và execution lifecycle. Không dùng như checklist lab; implementation cụ thể nằm ở Phần VI.

4.1. Spring Security nằm ở đâu trong request stack

- Spring Security Servlet support dựa trên Servlet Filters; nhiều filter có thể xử lý request trước DispatcherServlet.
- SecurityFilterChain là security boundary trước Controller, nên authentication/authorization không phải việc Controller tự làm bằng `if`.

4.2. SecurityContextHolder

- Giữ SecurityContext của execution hiện tại; code/framework có thể lấy Authentication hiện tại từ đó.
- Không tự dùng static/global user variable để lưu current user.

4.3. SecurityContext

- Chứa Authentication hiện tại. Authentication hợp lệ trở thành source of truth cho principal/authorities trong request.
- JWT filter thường tạo Authentication rồi đặt vào context sau khi token hợp lệ.

4.4. Authentication – hai vai trò

- Trước xác thực: object mang credential để đưa vào AuthenticationManager.
- Sau xác thực: object biểu diễn current authenticated user với principal + authorities; credential nhạy cảm thường được clear.

4.5. Principal

- Principal không nhất thiết là Entity. Trong username/password có thể là UserDetails; trong Resource Server JWT production có thể là Jwt object.
- Business code nên phụ thuộc identity contract cần thiết, tránh giả định framework object quá sâu nếu không cần.

4.6. GrantedAuthority và Role

- GrantedAuthority là high-level permission. Role chỉ là một convention/grouping.
- `roles("USER")` thường tạo `ROLE_USER`; `hasRole("USER")` kiểm role convention.

4.7. AuthenticationManager / ProviderManager

- AuthenticationManager định nghĩa cách filters/use case yêu cầu authentication.
- ProviderManager thường delegate tới một hoặc nhiều AuthenticationProvider phù hợp token type.

4.8. DaoAuthenticationProvider

- Dùng UserDetailsService load username/password/attributes và PasswordEncoder để kiểm credential.
- Đây là cầu nối quan trọng giữa database user và Spring Security username/password authentication.

4.9. UserDetailsService lifecycle

- Nhận username/email; query repository; trả UserDetails hoặc throw user-not-found.
- Không generate JWT trong UserDetailsService; tách lookup khỏi token issuance.

4.10. UserDetails fields

- username, password, authorities và account state (enabled/locked/expired).
- Course source nêu `getAuthorities()` phục vụ permissions và `getUsername()` trả email vì email unique.

4.11. Password storage

- Signup: raw password → `PasswordEncoder.encode` → encoded value → DB.
- Login: raw submitted password + DB hash → `matches`.
- Spring Security current docs khuyến nghị adaptive one-way functions và tune work factor theo hệ thống.

4.12. Vì sao không hash password bằng SHA-256 đơn giản

- Password hashing cần intentionally expensive + salt/work factor để chống offline cracking.
- General cryptographic hash rất nhanh và không phải password storage strategy phù hợp.

4.13. DelegatingPasswordEncoder awareness

- Hệ thống dài hạn có thể cần nhiều encoding scheme và migration; encoded password có prefix id kiểu `{bcrypt}` trong một số strategies.
- Không hard-code assumption rằng mọi password hash luôn cùng format mãi mãi.

4.14. JWT compact structure

- Header chứa metadata như alg/typ; payload chứa claims; signature bảo vệ integrity.
- JWT payload thường base64url encode chứ không encrypted.

4.15. JWT registered claims

- `sub` subject, `iat` issued-at, `exp` expiration; production thường quan tâm thêm `iss`, `aud`, `nbf`, `jti`.
- Course implementation tối thiểu subject + issuedAt + expiration + signature.

4.16. JWT creation

- Builder tạo claims/time, ký bằng key rồi compact.
- Signing key phải externalized; không nằm frontend hoặc repository.

4.17. JWT parsing/validation

- Parser/verifier kiểm signature trước khi tin claims. Sau đó kiểm expiration và các claims theo contract.
- Method `extractUsername()` chỉ an toàn khi underlying parse/validation strategy đúng.

4.18. Token lifetime

- Shorter token giảm blast radius nhưng tăng nhu cầu re-auth/refresh mechanism.
- Lab có thể dùng expiration đơn giản; production cần lifecycle strategy rõ.

4.19. JWT revocation limitation

- Self-contained JWT khó revoke tức thời nếu không có state phụ/introspection/denylist.

- Đây là trade-off chứ không phải bug riêng của Spring.

4.20. JwtAuthenticationFilter algorithm

- 1) đọc Authorization header; 2) kiểm `Bearer `; 3) extract token; 4) resolve subject; 5) load user nếu cần; 6) validate; 7) build Authentication; 8) set context; 9) continue chain.
- Nếu header không có bearer token, filter thường continue và authorization layer quyết định endpoint public/protected.

4.21. OncePerRequestFilter

- Giúp filter logic chạy một lần cho mỗi request dispatch phù hợp.
- Không có nghĩa mọi internal dispatcher/error case đều tuyệt đối một lần trong mọi configuration; sinh viên tập trung conceptual responsibility.

4.22. Filter order

- JWT filter phải đặt ở vị trí hợp lý trước username/password authentication filter baseline khi custom flow yêu cầu.
- Sai order có thể làm request không được authenticated trước authorization.

4.23. SecurityFilterChain rules

- `requestMatchers("/auth/**").permitAll()` cho signup/login; `anyRequest().authenticated()` bảo vệ còn lại.
- Rule rộng hoặc sai thứ tự có thể mở endpoint ngoài ý muốn.

4.24. Stateless session policy

- Custom JWT REST thường dùng `SessionCreationPolicy.STATELESS` để không dựa server HTTP session cho authentication.
- Stateless auth vẫn có DB user, config, key và business state.

4.25. CSRF scope awareness

- Lab bearer-token stateless thường disable CSRF, nhưng đây là architecture-dependent decision.
- Slot 26 sẽ hợp nhất CSRF/CORS/negative testing sâu hơn; không học thuộc `csrf.disable()` như universal rule.

4.26. AuthenticationService – signup

- Validate input/duplicate → encode password → assign server-controlled default role → save.
- Không cho client tự chọn ADMIN role khi public signup nếu policy không cho phép.

4.27. AuthenticationService – login

- Tạo UsernamePasswordAuthenticationToken từ email/password → AuthenticationManager.authenticate.
- Nếu success mới generate JWT. Không query DB + `password.equals(...)` thủ công.

4.28. AuthenticationController

- Nhận request DTO, gọi service, trả response DTO/token.
- Controller không nên trả User entity chứa encoded password.

4.29. `/users/me`

- Dùng current Authentication/Principal để trả identity hiện tại thay vì nhận userId tùy ý từ client.
- Đây là ví dụ quan trọng để hiểu SecurityContext được business API tiêu thụ thế nào.

4.30. `/users` và authorization direction

- Course slide có endpoint users; mức quyền cụ thể tùy source/project implementation.
- Nếu project áp role restriction, policy phải được viết rõ và test ở Part C.

4.31. 401/403 boundary awareness

- Authentication failure/missing credential thường dẫn 401; authenticated nhưng bị deny thường 403.
- Slot 25 chỉ cần hiểu boundary; Slot 26 sẽ xây negative matrix đầy đủ.

4.32. Error handling

- Invalid token/parser exception phải được chuyển thành auth failure hợp lý, không để 500 leak stacktrace.
- Login failure không nên tiết lộ quá chi tiết email tồn tại nhưng password sai trong production.

4.33. Testing boundaries

- Unit: JwtService/PasswordEncoder mapping. Integration: signup/login/token/protected request. DB verification: password hash.
- Happy-path test ở Slot 25; systematic negative/CORS tests ở Slot 26.

4.34. Current ecosystem bridge

- Spring Security current reference (retrieved Oct 2026) lists 7.1.1 and retains filter-based authentication architecture.
- OAuth2 Resource Server supports JWT/opaque token; current JWT support validates signature and standard time/issuer claims with issuer-based configuration.
- Course keeps Boot 3/Security 6 style to match syllabus; production radar is informational.

V. KINH NGHIỆM THỰC TIỄN, TIPS VÀ DEBUGGING

Vấn đề	Triệu chứng	Root cause thường gặp	Hướng xử lý
Login luôn fail	401	hash/encoder mismatch; user lookup sai	inspect DB hash + UserDetailsService + matches
Password lưu plaintext	DB thấy password thật	quên encoder ở signup	encode before save
Token generate nhưng protected 401	401	filter không chạy/key parse sai/context chưa set	breakpoint JwtFilter
Token parse 500	500	exception không map auth failure	catch/entry point strategy
Protected endpoint public	200 không token	permitAll/matcher sai	review SecurityFilterChain
Auth endpoint bị khóa	/auth/login 401	quên permitAll	fix matcher
Role không xuất hiện	403 later	UserDetails authorities mapping	inspect getAuthorities
Circular/coupled User entity	serialization/security coupling	Entity implements too much	DTO/adapter production direction
JWT secret trong Git	credential leak	hard-code config	externalize/rotate
Raw password/token log	sensitive leakage	debug logging careless	mask/remove

5.1. Debugging decision tree

```
SIGNUP FAILS
|
+-- validation?
+-- duplicate email?
+-- PasswordEncoder bean?
+-- DB constraint/datasource?
+-- password in DB encoded?

LOGIN FAILS
|
+-- user found?
+-- stored hash valid?
+-- AuthenticationManager wired?
+-- AuthenticationProvider uses correct UserDetailsService/PasswordEncoder?
+-- BadCredentials -> expected auth failure

PROTECTED REQUEST FAILS
|
+-- Authorization header starts "Bearer "?
+-- JwtAuthenticationFilter invoked?
+-- token parse/signature/expiration valid?
+-- UserDetails loaded?
+-- Authentication set in SecurityContext?
+-- endpoint authorization rule?
```

5.2. Tips

- Đừng viết JWT trước khi password authentication path chạy đúng.

- Tạo checkpoint sau mỗi layer: user lookup → password matches → token generation → token validation → context → controller.
- Không dùng entity response cho auth endpoint nếu nó chứa password hash.
- Role default phải do server quyết định trong public signup.
- Không log raw token/password để debug.
- Giữ JwtService thuần security/token concern; không nhét business CRUD vào đó.
- Current production system nên cân nhắc Resource Server/Authorization Server thay vì duy trì custom token stack lâu dài.

VI. HƯỚNG DẪN STEP BY STEP THỰC BÀI TẬP CHỦ ĐỀ VÀ BÀI TẬP TỔNG HỢP HỆ THỐNG

WORKBOOK RULE: Mỗi bước phải có Mục tiêu → thao tác → code/config → lệnh chạy → verification → PASS/FAIL.

6.1. Step 1 – Tạo/kiểm project dependencies

Mục tiêu: project có đúng Web/Security/JPA/SQL Server/Test và JWT library.

Windows check

```
mvn dependency:tree | findstr /I "security data-jpa mssql jjwt"
# PowerShell có thể dùng Select-String tương đương
```

PASS: dependency tree có Spring Security + JPA + SQL Server + JWT library theo Lab 06.

SOURCE: Chapter 16/Lab 06 dùng JJWT (`io.jsonwebtoken.Jwts`) để tạo/validate token.

6.2. Step 2 – Cấu hình datasource và JWT settings

application.properties

```
spring.datasource.url=jdbc:sqlserver://localhost:1433;databaseName=OrchidManagementDB;encrypt=true;trustServer
Certificate=true
spring.datasource.username=sa
spring.datasource.password=${DB_PASSWORD}

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

security.jwt.secret=${JWT_SECRET}
security.jwt.expiration-ms=900000
```

Thao tác: set environment variables trong Run Configuration/OS; không ghi secret thật vào file commit.

PASS: application start + datasource connected; config resolves.

6.3. Step 3 – Tạo User entity

Course-ready entity

```
@Entity
@Table(name = "AppUser")
public class AppUser {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 120)
    private String email;

    @Column(nullable = false, length = 255)
    private String password;

    @Column(nullable = false, length = 30)
    private String role = "USER";
```

```
    @Column(nullable = false)
    private boolean enabled = true;

    // getters/setters
}
```

Nếu bám sát Lab 06, entity có thể implements `UserDetails`; tài liệu này tách entity và adapter/user details ở bước sau để trách nhiệm rõ hơn.

SQL verification

```
SELECT TOP (100) *
FROM dbo.AppUser
ORDER BY id DESC;
```

PASS: Hibernate tạo/match table; app start không mapping error.

6.4. Step 4 – UserRepository

```
public interface AppUserRepository extends JpaRepository<AppUser, Long> {
    Optional<AppUser> findByEmail(String email);
    boolean existsByEmail(String email);
}
```

Verification: viết repository test hoặc dùng service lookup; không cần tự implement CRUD.

6.5. Step 5 – DTO và response classes

```
public record SignupRequest(
    @Email @NotBlank String email,
    @Size(min = 8, max = 100) String password
) {}

public record LoginRequest(
    @Email @NotBlank String email,
    @NotBlank String password
) {}

public record AuthResponse(
    String accessToken,
    String tokenType,
    long expiresIn
) {}

public record UserResponse(
    Long id,
    String email,
    String role
) {}
```

SECURITY: Không đưa password/passwordHash vào UserResponse.

6.6. Step 6 – PasswordEncoder

```
@Bean
PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

Local learning experiment

```
PasswordEncoder encoder = new BCryptPasswordEncoder();
String hash = encoder.encode("Sba301@2026");
System.out.println(encoder.matches("Sba301@2026", hash)); // true
```

PASS: hash không bằng raw password; matches trả true.

6.7. Step 7 – UserDetailsService

```
@Bean
UserDetailsService userDetailsService(AppUserRepository users) {
    return username -> {
        AppUser u = users.findByEmail(username)
            .orElseThrow(() -> new UsernameNotFoundException("User not found"));

        return User.withUsername(u.getEmail())
            .password(u.getPassword())
            .roles(u.getRole())
            .disabled(!u.isEnabled())
            .build();
    };
}
```

Đặt breakpoint trong lambda/method và gọi login sau này để chứng minh repository lookup xảy ra.

SOURCE: Course slide/Lab 06 xem email là username unique và `getAuthorities()` là permission source.

6.8. Step 8 – AuthenticationProvider và AuthenticationManager

```
@Bean
AuthenticationProvider authenticationProvider(
        UserDetailsService userDetailsService,
        PasswordEncoder passwordEncoder) {

    DaoAuthenticationProvider provider =
        new DaoAuthenticationProvider(userDetailsService);
    provider.setPasswordEncoder(passwordEncoder);
    return provider;
}

@Bean
AuthenticationManager authenticationManager(
        AuthenticationConfiguration configuration) throws Exception {
    return configuration.getAuthenticationManager();
}
```

VERSION: Spring Security API signatures có thể thay đổi theo minor/current version; compile theo version project và xem official reference nếu constructor/setter khác.

6.9. Step 9 – Signup service

```
@Transactional
public UserResponse signup(SignupRequest request) {
    if (users.existsByEmail(request.email())) {
        throw new IllegalStateException("EMAIL_EXISTS");
    }

    AppUser user = new AppUser();
    user.setEmail(request.email());
    user.setPassword(passwordEncoder.encode(request.password()));
    user.setRole("USER");

    AppUser saved = users.save(user);
    return new UserResponse(saved.getId(), saved.getEmail(), saved.getRole());
}
```

Postman

```
POST http://localhost:8080/auth/signup
Content-Type: application/json

{
```

```
  "email": "slot25.student@example.com",
  "password": "Sba301@2026"
}
```

Verification: 200/201 theo contract; SQL DB lưu encoded password, không plaintext.

```
SELECT id, email, password, role, enabled
FROM dbo.AppUser
WHERE email = 'slot25.student@example.com';
```

6.10. Step 10 – JWT signing key helper và JwtService

Ví dụ dưới đây theo tinh thần JJWT của Chapter 16/Lab 06. API cụ thể phụ thuộc JJWT version trong project.

JwtService skeleton

```
@Service
public class JwtService {
    @Value("${security.jwt.secret}")
    private String secret;

    @Value("${security.jwt.expiration-ms}")
    private long expirationMs;

    private SecretKey signingKey() {
        return Keys.hmacShaKeyFor(Decoders.BASE64.decode(secret));
    }

    public String generateToken(UserDetails user) {
        Instant now = Instant.now();
        return Jwts.builder()
            .subject(user.getUsername())
            .issuedAt(Date.from(now))
            .expiration(Date.from(now.plusMillis(expirationMs)))
            .signWith(signingKey())
            .compact();
    }

    public Claims parseClaims(String token) {
        return Jwts.parser()
            .verifyWith(signingKey())
            .build()
            .parseSignedClaims(token)
            .getPayload();
    }

    public String extractUsername(String token) {
        return parseClaims(token).getSubject();
    }

    public boolean isValid(String token, UserDetails user) {
        Claims claims = parseClaims(token);
        return claims.getSubject().equals(user.getUsername())
            && claims.getExpiration().after(new Date());
    }
}
```

VERSION: Không copy API nếu JJWT version khác; mục tiêu bắt buộc là generate + signature validation + claim extraction + expiration check.

6.11. Step 11 – Login service qua AuthenticationManager

```
public AuthResponse login(LoginRequest request) {
    Authentication authentication =
        authenticationManager.authenticate(
```

```
            new UsernamePasswordAuthenticationToken(
                request.email(), request.password()));

    UserDetails principal = (UserDetails) authentication.getPrincipal();
    String token = jwtService.generateToken(principal);
    return new AuthResponse(token, "Bearer", expirationMs / 1000);
}
```

Checkpoint: đặt breakpoint tại `authenticate(...)`; xem call stack sang provider/UserDetailsService/PasswordEncoder.

Postman

```
POST http://localhost:8080/auth/login
Content-Type: application/json

{
  "email": "slot25.student@example.com",
  "password": "Sba301@2026"
}
```

PASS: 200 + token. Sai password: authentication failure (không 500).

6.12. Step 12 – JwtAuthenticationFilter

```
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {
    private final JwtService jwtService;
    private final UserDetailsService userDetailsService;

    public JwtAuthenticationFilter(JwtService jwtService,
                                   UserDetailsService userDetailsService) {
        this.jwtService = jwtService;
        this.userDetailsService = userDetailsService;
    }

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain)
            throws ServletException, IOException {

        String header = request.getHeader("Authorization");
        if (header == null || !header.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
            return;
        }

        String token = header.substring(7);
        try {
            String username = jwtService.extractUsername(token);
            if (username != null && SecurityContextHolder.getContext()
                    .getAuthentication() == null) {
                UserDetails user = userDetailsService.loadUserByUsername(username);
                if (jwtService.isValid(token, user)) {
                    UsernamePasswordAuthenticationToken auth =
                        new UsernamePasswordAuthenticationToken(
                            user, null, user.getAuthorities());
                    auth.setDetails(new WebAuthenticationDetailsSource()
                        .buildDetails(request));
                    SecurityContextHolder.getContext().setAuthentication(auth);
                }
            }
        } catch (JwtException | IllegalArgumentException ex) {
            SecurityContextHolder.clearContext();
```

```
        }
        filterChain.doFilter(request, response);
    }
}
```

PASS: filter compiles; no raw token logging.

6.13. Step 13 – SecurityFilterChain

```
@Bean
SecurityFilterChain securityFilterChain(HttpSecurity http,
        JwtAuthenticationFilter jwtFilter) throws Exception {

    http
        .csrf(csrf -> csrf.disable())
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/auth/**").permitAll()
            .anyRequest().authenticated())
        .sessionManagement(sm -> sm
            .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authenticationProvider(authenticationProvider)
        .addFilterBefore(jwtFilter,
            UsernamePasswordAuthenticationFilter.class);

    return http.build();
}
```

SCOPE: CORS-specific consolidation và negative matrix được triển khai ở Slot 26 Part C.

6.14. Step 14 – AuthenticationController

```
@RestController
@RequestMapping("/auth")
public class AuthenticationController {
    private final AuthService authService;

    @PostMapping("/signup")
    public ResponseEntity<UserResponse> signup(
            @Valid @RequestBody SignupRequest request) {
        return ResponseEntity.status(HttpStatus.CREATED)
            .body(authService.signup(request));
    }

    @PostMapping("/login")
    public ResponseEntity<AuthResponse> login(
            @Valid @RequestBody LoginRequest request) {
        return ResponseEntity.ok(authService.login(request));
    }
}
```

Verification: signup/login hoạt động dù chưa có token vì `/auth/**` permitAll.

6.15. Step 15 – Protected `/users/me` endpoint

```
@RestController
@RequestMapping("/users")
public class UserController {
    @GetMapping("/me")
    public ResponseEntity<Map<String,Object>> me(
            Authentication authentication) {
        return ResponseEntity.ok(Map.of(
            "username", authentication.getName(),
            "authorities", authentication.getAuthorities()));
```

```
    }
}
```

Postman

```
GET http://localhost:8080/users/me
Authorization: Bearer {{access_token}}
```

PASS: 200 + username/authorities; không truyền userId từ client để biết `me`.

6.16. Step 16 – Protected `/users` endpoint

Chapter 16 demo có `GET /users`. Ở baseline Slot 25 chỉ chứng minh protected endpoint; role policy chi tiết sẽ được khóa ở Slot 26.

```
@GetMapping
public List<UserResponse> findAll() {
    return users.findAll().stream()
        .map(u -> new UserResponse(u.getId(), u.getEmail(), u.getRole()))
        .toList();
}
```

PASS: không token → security reject; valid token → response theo current rule.

6.17. Step 17 – Authentication error handling baseline

Chapter 16/Lab 06 có bước customize authentication error messages. Tạo entry point đơn giản để không trả HTML/default stack.

```
@Component
public class RestAuthenticationEntryPoint implements AuthenticationEntryPoint {
    @Override
    public void commence(HttpServletRequest request,
                         HttpServletResponse response,
                         AuthenticationException ex) throws IOException {
        response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
        response.setContentType("application/json");
        response.getWriter().write("{\"error\":\"UNAUTHORIZED\"}");
    }
}
```

```
http.exceptionHandling(ex ->
    ex.authenticationEntryPoint(restAuthenticationEntryPoint));
```

PASS: protected no-token/invalid-auth returns JSON 401 rather than 500.

6.18. Step 18 – Trace login end-to-end

Đặt breakpoint tại:

- AuthenticationController.login
- AuthService.login
- AuthenticationManager.authenticate call site
- UserDetailsService.loadUserByUsername
- UserRepository.findByEmail
- PasswordEncoder.matches path
- JwtService.generateToken

Trace expected

```
POST /auth/login
 -> Controller.login
 -> AuthService.login
 -> AuthenticationManager.authenticate
 -> DaoAuthenticationProvider
 -> UserDetailsService.loadUserByUsername
 -> UserRepository.findByEmail
 -> PasswordEncoder.matches
 <- authenticated UserDetails
```

```
 -> JwtService.generateToken
 <- AuthResponse
```

6.19. Step 19 – Trace protected request end-to-end

- JwtAuthenticationFilter.doFilterInternal
- JwtService.extractUsername/parseClaims
- UserDetailsService.loadUserByUsername
- SecurityContextHolder.setAuthentication
- UserController.me

Trace expected

```
GET /users/me + Bearer JWT
 -> SecurityFilterChain
 -> JwtAuthenticationFilter
 -> JwtService.parse/validate
 -> UserDetailsService
 -> build Authentication
 -> SecurityContext
 -> authorization
 -> UserController.me
```

6.20. Step 20 – Minimal verification matrix

ID	Test	Expected	DB/Debug evidence
T01	Signup valid	201	new row; password encoded
T02	Signup duplicate	4xx contract	no duplicate row
T03	Login valid	200 + token	provider/matches success
T04	Login wrong password	401/auth failure	no token
T05	/users/me no token	401	controller not executed
T06	/users/me valid token	200	principal correct
T07	/users valid token	200 under baseline	protected rule works

SCOPE: Tampered/expired/role/CORS exhaustive negative tests được chuyển sang Slot 26 Part C để tránh trùng nội dung.

6.21. Step 21 – Automated test bridge

```
@SpringBootTest
@AutoConfigureMockMvc
class AuthFlowIntegrationTest {
    @Autowired MockMvc mvc;

    @Test
    void meWithoutToken_returns401() throws Exception {
        mvc.perform(get("/users/me"))
           .andExpect(status().isUnauthorized());
    }

    // TODO:
    // signup -> DB encoded password
    // login -> token
    // protected with real token -> 200
}
```

Mục tiêu Slot 25: tạo bridge sang regression tests; Slot 26 mở rộng negative matrix.

6.22. Bài tập tổng hợp hệ thống

1. Tạo user signup với validation và BCrypt.
2. Đăng nhập bằng AuthenticationManager/Provider.
3. Sinh JWT có subject/iat/exp và signature.
4. Xây JwtAuthenticationFilter và set SecurityContext.
5. Cấu hình `/auth/**` public, phần còn lại protected.
6. Tạo `/users/me` dùng current Authentication.

7. Tạo `/users` protected baseline.
8. Customize authentication error JSON.
9. Trace login + protected request bằng debugger.
10. Viết README: security flow, endpoints, config, known limitations, production direction.

6.23. Integration Gate G1–G12

Gate	Evidence
G1 Build	compile/test/start pass
G2 User persistence	user row + encoded password
G3 Signup	valid + duplicate handling
G4 UserDetails	email lookup + authorities
G5 AuthManager	login uses provider, not manual equals
G6 JWT issue	token returned after successful auth
G7 JWT validate	valid token parses/validates
G8 Filter	valid JWT creates Authentication
G9 Context	/users/me sees principal
G10 Protection	protected endpoint no token → 401
G11 Trace	login + request chains documented
G12 Explain	human verification: each bean responsibility

VII. TỪ ĐIỂN THUẬT NGỮ – FOUNDATION GLOSSARY

Thuật ngữ	Định nghĩa	Ví dụ Slot 25
Authentication	Xác minh identity	login credential verification
Authorization	Quyết định quyền	endpoint access
Principal	Identity đã xác thực	UserDetails/email
GrantedAuthority	Permission high-level	ROLE_USER
Role	Nhóm permission	USER/ADMIN
SecurityContextHolder	Truy cập SecurityContext hiện tại	current Authentication
SecurityContext	Chứa Authentication	request security context
Authentication	Credential input / authenticated identity	UsernamePasswordAuthenticationToken
AuthenticationManager	Điều phối authentication	authenticate(...)
AuthenticationProvider	Strategy xác thực	DaoAuthenticationProvider
DaoAuthenticationProvider	Username/password provider	UserDetailsService + PasswordEncoder
UserDetailsService	Load security user	find by email
UserDetails	Security-facing user	username/password/authorities
PasswordEncoder	Encode/matches password	BCryptPasswordEncoder
SecurityFilterChain	Security filters + rules	HttpSecurity
OncePerRequestFilter	Base filter class	JwtAuthenticationFilter
Bearer Token	Credential trong Authorization header	Bearer eyJ...
JWT	JSON Web Token	header.payload.signature
Claim	Dữ liệu trong payload	sub/iat/exp
Signing Key	Key ký/verify JWT	HMAC key
Expiration	Token validity end	exp
Stateless	Không dựa HTTP session auth state	STATELESS
AuthenticationEntryPoint	Handle unauthenticated request	401 JSON
Auth DTO	Request/response security boundary	LoginRequest/AuthResponse

VIII. RESEARCH BASE VÀ AI-BASED LEARNING – 4 NHÓM TRẢI NGHIỆM CUỐI BUỔI

8.1. Quy tắc AI

- Không đưa password, JWT secret/private key hoặc token thật vào AI prompt.
- AI chỉ được phân tích class/method có source; nếu thiếu file phải nói thiếu bằng chứng.
- Execution claim phải được kiểm bằng debugger/call stack/log.
- Current recommendation phải kiểm official Spring Security docs/RFC.
- Phân biệt Course Baseline custom JWT và Production Direction Resource Server.

8.2. Nhóm I – Workflow / execution rules

PROMPT I.1: Trace login request của source tôi: Controller → Service → AuthenticationManager → Provider → UserDetailsService → Repository → PasswordEncoder → JwtService. Với mỗi bước ghi class/method/input/output.

PROMPT I.2: Trace protected request Bearer JWT từ SecurityFilterChain → JwtAuthenticationFilter → JwtService → UserDetailsService → SecurityContext → Controller.

PROMPT I.3: Phân tích signup flow và chỉ ra nơi raw password tồn tại, nơi encoded password được tạo, nơi raw password không được xuất hiện.

PROMPT I.4: Giải thích Authentication object trước và sau authentication trong project; chỉ ra breakpoint quan sát `isAuthenticated`, principal và authorities.

PROMPT I.5: Phân tích SecurityContext được set/clear ở đâu và Controller lấy current user thế nào.

PROMPT I.6: Trace bad-password login và chỉ ra exception/outcome tại provider/manager.

PROMPT I.7: Trace request không có Authorization header qua JWT filter; tại sao filter vẫn continue chain.

PROMPT I.8: Vẽ bean dependency graph AuthController/AuthService/AuthenticationManager/Provider/UserDetailsService/Repository/PasswordEncoder/JwtService.

PROMPT I.9: Phân tích startup flow tạo SecurityFilterChain và filter registration.

PROMPT I.10: Tạo 12 câu hỏi debugger-driven để sinh viên tự chứng minh authentication architecture.

8.3. Nhóm II – 10 prompt kỹ thuật nâng cao

PROMPT II.1: So sánh Entity implements UserDetails với security adapter riêng. Phân tích coupling, serialization, testing và domain purity.

PROMPT II.2: So sánh BCrypt, Argon2, PBKDF2 và DelegatingPasswordEncoder trong Spring Security; tạo benchmark plan tune work factor.

PROMPT II.3: Phân tích AuthenticationManager/ProviderManager khi có nhiều AuthenticationProvider. Provider selection/supports() hoạt động ra sao?

PROMPT II.4: Phân tích credential erasure sau authentication và lý do giảm leakage risk.

PROMPT II.5: Phân tích JWT claim design: subject, role/authorities, issuer, audience, jti; claim nào course cần và claim nào production cần.

PROMPT II.6: So sánh HMAC và asymmetric JWT signing khi cùng app issue/consume token vs tách Authorization Server/Resource Server.

PROMPT II.7: Phân tích filter ordering và 8 failure modes nếu JWT filter đặt sai chỗ hoặc set context sai.

PROMPT II.8: Review custom JwtAuthenticationFilter về parser exceptions, context overwrite, duplicate DB lookup và authority trust.

PROMPT II.9: Phân tích token lifetime/revocation trade-off; thiết kế experiment expired token nhưng không chuyển thành Part C negative matrix.

PROMPT II.10: Thiết kế method-level authorization extension (`@PreAuthorize`) và so sánh với URL-level authorization về separation of concerns.

8.4. Nhóm III – 10 prompt production engineering

PROMPT III.1: Review custom JWT lab như Security Architect và lập migration map sang OAuth2 Resource Server: class nào bỏ/giữ/thay.

PROMPT III.2: Thiết kế authentication architecture cho monolith vs microservices: login/token issuer/resource server boundaries.

PROMPT III.3: Thiết kế secret/key management cho JWT qua dev/staging/prod và rotation/incident response.

PROMPT III.4: Thiết kế user-account lifecycle: signup, email verification, password reset, lockout, disable, credential rotation.

PROMPT III.5: Thiết kế login abuse protection: rate limiting, failed-attempt metrics, lockout trade-offs và enumeration resistance.

PROMPT III.6: Thiết kế consistent authentication error response mà không tiết lộ user tồn tại hay không.

PROMPT III.7: Thiết kế audit events cho signup/login/authentication failure/token issuance mà không log secrets.

PROMPT III.8: Thiết kế CI security regression tests cho authentication pipeline + real JWT path.

PROMPT III.9: Thiết kế database indexes/constraints cho email lookup và account uniqueness dưới concurrent signup.

PROMPT III.10: Thiết kế migration từ single `role` string sang normalized permission/role model khi hệ thống lớn dần.

8.5. Nhóm IV – 10 prompt current ecosystem/community trends

PROMPT IV.1: Kiểm tra Spring Security stable hiện tại và mô tả Authentication architecture hiện hành: SecurityContextHolder, AuthenticationManager, Provider, UserDetailsService.

PROMPT IV.2: Kiểm tra current Spring Security Password Storage guidance: BCrypt/Argon2/work-factor tuning và DelegatingPasswordEncoder.

PROMPT IV.3: Kiểm tra current OAuth2 Resource Server JWT support và lập bảng custom JwtFilter vs framework Resource Server.

PROMPT IV.4: Kiểm tra current Spring Security Bearer token principal/authority mapping và `SCOPE_` convention.

PROMPT IV.5: Khảo sát Spring Authorization Server current state và use case khi tách token issuance khỏi business API.

PROMPT IV.6: Kiểm tra JJWT current API/version migration notes cho builder/parser/signWith/verifyWith; so sánh với course snippets.

PROMPT IV.7: Khảo sát JWT key rotation/JWK/JWKS best practice từ Spring/RFC sources và giải thích tác động architecture.

PROMPT IV.8: Khảo sát passkeys/WebAuthn/MFA như authentication trends; phân biệt credential authentication với API authorization token.

PROMPT IV.9: Khảo sát OWASP Authentication Cheat Sheet cập nhật và map 10 recommendation vào signup/login implementation.

PROMPT IV.10: Tạo 2026 Authentication/JWT Technology Radar cho Spring REST: MUST KNOW NOW / SHOULD EXPLORE / WATCH LATER, có URL/date evidence.

8.6. AI Verification Worksheet

Claim	Nguồn kiểm chứng	Experiment	Kết luận
Authentication chain	source + debugger	login breakpoints	
Password hashing	Spring docs + DB	encode/matches	
JWT validation	source + unit test	valid token	
Filter order	source + call stack	protected GET	
Production Resource Server	official docs	minimal comparison	

IX. SINH VIÊN CẦN TỰ KIỂM TRA GÌ SAU BUỔI HỌC

□ 1. Tôi giải thích được Authentication và Authorization bằng hai câu hỏi khác nhau.
□ 2. Tôi biết Principal, Authority và Role nằm ở đâu trong Authentication.
□ 3. Tôi mô tả được AuthenticationManager và AuthenticationProvider khác nhau thế nào.
□ 4. Tôi giải thích được UserDetailsService và Repository nối với nhau ra sao.
□ 5. Tôi biết PasswordEncoder.encode và matches dùng ở bước nào.
□ 6. Tôi không lưu hoặc trả raw password/password hash ra API response.
□ 7. Tôi tạo JWT chỉ sau authentication thành công.
□ 8. Tôi phân biệt decode JWT và validate JWT.
□ 9. Tôi trace được JwtAuthenticationFilter từng bước.
□ 10. Tôi hiểu khi nào SecurityContext được set.
□ 11. Tôi có thể tạo `/users/me` từ current Authentication.
□ 12. Tôi biết vì sao `/auth/**` phải permitAll nhưng protected endpoints không được mở.
□ 13. Tôi có Postman evidence signup/login/me.
□ 14. Tôi có DB evidence password encoded.
□ 15. Tôi có debugger evidence cho login và protected request.
□ 16. Tôi nêu được limitation của custom JWT lab và production Resource Server direction.

IX.1. Exit ticket 5 phút

Câu hỏi	Tự trả lời
AuthenticationManager làm gì?	
DaoAuthenticationProvider cần hai dependency cốt lõi nào?	
JWT được tạo trước hay sau password authentication?	
JWT filter set object gì vào SecurityContext?	
Một bằng chứng bạn đã tự kiểm bằng debugger hôm nay?	

TÀI LIỆU THAM CHIẾU & SOURCE TRACEABILITY

- Course source: Chapter 16 – Securing REST Services with Spring Boot, pp. 17–40: Spring Security terminology, Authentication/Authorization, JWT, implementation steps, signup/login/protected endpoints.

- Course source: Lab 06 – Secure REST Services with Spring Boot using JWT, pp. 10–29: User entity/UserDetails, DTO, repository, JWT classes, security configuration, filter, authentication service/controller, endpoints and Postman tests.
- Official Spring Security Reference – Servlet Architecture and Authentication Architecture.
- Official Spring Security Reference – UserDetailsService and PasswordEncoder/Password Storage.
- Official Spring Security Reference – OAuth2 Resource Server JWT/opaque bearer token support (production direction).

NOTE: Current ecosystem note: official Spring Security reference retrieved in October 2026 identifies 7.1.1 as current stable. This document intentionally retains Spring Boot 3.x / Spring Security 6.x course baseline for implementation compatibility; current-version differences are handled as research/production direction rather than silently changing the lab.